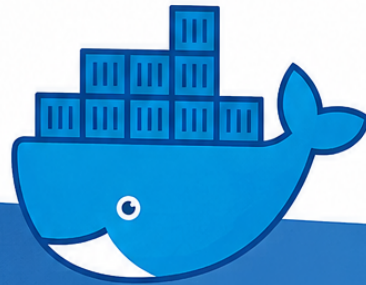


Salvatore D'Angelo



Getting Started with Docker™

A practical guide to containerize
and deploy modern applications



Understand
Containers



Build
Images



Run &
Deploy



Best Practices
& Tips



From zero to containerized.
Your journey starts here.

To every developer who has ever typed
`docker run hello-world`
and wondered what actually just happened.

Contents

Preface	8
Who This Book Is For	8
How This Book Is Structured	8
A Note on Docker vs Podman	9
Source Code and Updates	9
1 Getting Started with Docker and Podman	10
1.1 Introduction	10
1.2 Why Docker?	10
1.3 Docker vs Podman: Which One Should You Use?	11
1.4 Installing Docker or Podman	12
1.4.1 Docker	12
1.4.2 Podman (Mac)	12
1.5 Core Concepts: Images and Containers	13
1.5.1 Docker Image	13
1.5.2 Docker Container	13
1.6 Essential Commands	13
1.6.1 List local images	14
1.6.2 Pull an image from Docker Hub	14
1.6.3 Run a container	14
1.6.4 List running containers	15
1.6.5 View container logs	15
1.6.6 Execute a command inside a running container	15
1.6.7 Stop and remove a container	15
1.6.8 Remove an image	15
1.6.9 Clean up everything unused	16
1.7 Your First Containerized Application	16
1.8 How This Series Is Structured	17

1.9	Conclusion	17
2	Containers vs Virtual Machines	19
2.1	Introduction	19
2.2	Virtual Machines	19
2.2.1	Pros and cons of VMs	20
2.3	Containers	21
2.3.1	Pros and cons of containers	22
2.4	The Analogy	23
2.5	Containers and VMs: not enemies, complementaries	23
2.6	Conclusion	23
3	Dockerfile and Building Custom Images	24
3.1	Introduction	24
3.2	What is a Dockerfile?	24
3.3	Dockerfile Instructions	24
3.3.1	FROM	24
3.3.2	COPY	25
3.3.3	ADD	25
3.3.4	RUN	25
3.3.5	ENV	26
3.3.6	EXPOSE	26
3.3.7	CMD	26
3.3.8	ENTRYPOINT	26
3.3.9	USER	26
3.4	Building a Custom Nginx Image	27
3.5	Understanding the Layer Cache	28
3.6	Tagging and Versioning Images	29
3.7	Cleaning Up	29
3.8	Conclusion	29
4	How Docker Networking Works	31
4.1	Introduction	31
4.2	Networking Overview in Docker	31
4.2.1	The bridge driver	32
4.2.2	The host driver	32
4.2.3	The overlay driver	33

4.2.4	The macvlan driver	33
4.3	Docker Networking Commands	33
4.4	Practical Example: Frontend and Backend on the Same Network	34
4.4.1	Create the network	34
4.4.2	Start the “backend” container	34
4.4.3	Start the “frontend” container	35
4.4.4	Verify containers can resolve each other by name	35
4.4.5	Clean up	36
4.5	Conclusion	36
5	How Docker Volumes Work	37
5.1	Introduction	37
5.2	The Upgrade Problem	37
5.3	Docker Storage Types	37
5.3.1	Volumes	38
5.3.2	Bind Mounts	38
5.3.3	Tmpfs Mounts	39
5.4	The --mount Syntax	39
5.5	Docker Volumes Cheat Sheet	40
5.6	Practical Example: Nginx with Persistent Content	41
5.6.1	Create the volume	41
5.6.2	Populate the volume	41
5.6.3	Serve the content with Nginx	41
5.6.4	Replace the container — data survives	41
5.6.5	Clean up	42
5.7	Volumes vs Bind Mounts: When to Use Which	42
5.8	Conclusion	42
6	How Docker Compose Works	43
6.1	Introduction	43
6.2	What is Docker Compose?	43
6.3	The docker-compose.yml File	44
6.3.1	A Note on the version Field	44
6.4	Practical Example: Frontend and Backend with Compose	44
6.4.1	Project structure	45
6.4.2	The docker-compose.yml file	46

6.5	Docker Compose Commands	47
6.6	Environment Variables	48
6.7	Starting the Application	48
6.7.1	Clean up	49
6.8	Conclusion	49
7	Docker Security Best Practices	50
7.1	Introduction	50
7.2	1. Never Run as Root	50
7.3	2. Use a Read-Only Filesystem	52
7.4	3. Drop Linux Capabilities	53
7.5	4. Never Put Secrets in Environment Variables (or Images)	53
7.6	5. Use Minimal Base Images	54
7.7	6. Scan Images for Known Vulnerabilities	55
7.8	7. Keep Images Up to Date	55
7.9	Putting It All Together	57
7.10	Conclusion	58
8	Docker Image Best Practices	59
8.1	Introduction	59
8.2	1. Functional: Make the Image Behave Predictably	59
8.2.1	Use the exec form for CMD and ENTRYPOINT	59
8.2.2	Combine ENTRYPOINT and CMD	60
8.2.3	Declare a HEALTHCHECK	60
8.2.4	Use ENV for configuration, not for secrets	61
8.2.5	Add a .dockerignore	61
8.3	2. Lightweight: Ship Only What You Need	61
8.3.1	Start from a minimal base image	61
8.3.2	Use multi-stage builds	62
8.3.3	Chain RUN instructions and clean up in the same layer	63
8.3.4	Beware of chown -R in its own layer	63
8.3.5	Order instructions from least to most frequently changing	64
8.4	3. Debuggable: Make Failures Easy to Diagnose	64
8.4.1	Log to stdout/stderr, not to files	65
8.4.2	Keep a shell available, or know how to add one temporarily	65
8.4.3	Add LABEL metadata	65

8.4.4 Always use explicit, meaningful tags	65
8.5 4. Secure: Don't Undo Your Own Work	66
8.6 Putting It All Together	66
8.7 Checklist	67
8.8 Conclusion	68
Glossary	69
What's Next?	71
Where to Go From Here	71
Docker Swarm	71
Kubernetes	71
CI/CD with Docker and GitHub Actions	72
Docker Security (deeper)	72
Stay in Touch	72

Preface

I started working with Docker in 2018 when containers were already gaining momentum but most tutorials either assumed too much prior knowledge or buried the practical steps under layers of theory.

This book is what I wish I had back then.

It is a no-nonsense, hands-on introduction to Docker and Podman. Every chapter introduces one concept and immediately applies it with a concrete example. By the time you finish the last chapter, you will have built, networked, persisted, orchestrated, and secured a real multi-container application — step by step, command by command.

Who This Book Is For

This book is for developers who:

- Have heard of Docker but have never used it in practice
- Want to understand the fundamentals before jumping into Kubernetes
- Work on a corporate machine where Docker Desktop requires a paid license and need a free alternative (Podman)
- Want a single, coherent reference instead of scattered blog posts

No prior knowledge of containers or virtualization is assumed. Basic familiarity with the command line is all you need.

How This Book Is Structured

Each chapter of this book corresponds to a foundational Docker concept, introduced in a deliberate order so that each one builds on the previous:

1. **Getting Started with Docker and Podman** — Learn what Docker is, understand containers and images, and run your first containerized web server with Docker or Podman

-
- no setup complexity required.
 - 2. **Containers vs Virtual Machines** — Understand the real differences between containers and virtual machines — how each one works under the hood, their pros and cons, and when to use which.
 - 3. **Dockerfile and Building Custom Images** — Learn how to write a Dockerfile, build a custom image, understand the layer cache, and serve your own web content with a custom Nginx container.
 - 4. **How Docker Networking Works** — Learn how Docker networking works — bridge, host, overlay, and macvlan drivers — and how to connect containers so they can communicate over a private network.
 - 5. **How Docker Volumes Work** — Learn the three Docker storage types — volumes, bind mounts, and tmpfs — and how to use them to persist data outside a container so it survives restarts and replacements.
 - 6. **How Docker Compose Works** — Learn how Docker Compose v2 works — define, build, and run multi-container applications with a single YAML file and a single command.
 - 7. **Docker Security Best Practices** — Learn how to run Docker containers securely — non-root users, read-only filesystems, capability limits, secret management, and image vulnerability scanning.
 - 8. **Docker Image Best Practices** — A practical checklist for writing Dockerfiles that produce functional, lightweight, debuggable, and secure images — multi-stage builds, layer ordering, healthchecks, and more.

A Note on Docker vs Podman

All commands in this book are shown with the `docker` prefix. If you are using Podman, every command works identically — simply replace `docker` with `podman`, or set the alias `alias docker=podman` once and never think about it again.

Source Code and Updates

The examples in this book are maintained on GitHub at github.com/sasadangelo/docker-tutorials.

For corrections, suggestions, or feedback, write to sasadangelo@gmail.com or visit code4projects.org.

Chapter 1

Getting Started with Docker and Podman

1.1 Introduction

This is the first article of the **Getting Started with Docker** series. By the end of this article you will understand what Docker is, what problems it solves, and you will have your first containerized web server running on your machine in minutes — without writing a single line of Dockerfile.

You should read this article if:

- You have heard of Docker but never used it
- You want a practical, no-nonsense introduction that gets you running immediately
- You are on a machine where Docker Desktop is not available and you need an alternative (Podman)

All the commands shown here work identically with both **Docker** and **Podman**. Where there are differences, they are explicitly noted.

1.2 Why Docker?

Before diving into commands, it is worth spending a minute on the problem Docker actually solves.

Imagine you are a developer. You write an application on your laptop, it works perfectly. Then you hand it off to a colleague or deploy it to a server — and it breaks. Different OS version, different library installed, different path. This is the classic “**it works on my machine**” problem,



Figure 1.1: Getting Started with Docker and Podman

and it has tormented developers for decades.

Docker solves this by packaging your application together with everything it needs to run — runtime, libraries, configuration — into a single portable unit called a **container**. That container runs identically on your laptop, your colleague’s machine, and the production server.

This matters not just for individual developers. Modern applications are built as dozens (sometimes hundreds) of independent services — **microservices** — each with its own dependencies. Docker makes it possible to deploy, update, and scale each service independently, without interference.

1.3 Docker vs Podman: Which One Should You Use?

Both Docker and Podman are tools for running containers. For everything covered in this series, they are interchangeable — the CLI commands are identical.

The key differences:

	Docker	Podman
Architecture	Requires a background daemon (dockerd)	Daemonless — runs containers directly

	Docker	Podman
Privileges	Daemon runs as root	Rootless by default — more secure
License	Docker Desktop requires a paid license for enterprise use	Fully open source (Apache 2.0)
Installation on Mac	Docker Desktop	<code>brew install podman + podman machine init</code>
Compose support	<code>docker compose</code> (v2 plugin)	<code>podman compose</code> (from Podman 4.7+)
OCI compatibility	Yes	Yes — same image format

In short: if Docker Desktop is available and licensed for your use, use Docker. If you are on a corporate machine where Docker Desktop requires a commercial license, or if you prefer an open source rootless alternative, use Podman. The rest of this series works exactly the same either way.

Throughout this series, all commands are shown with `docker`. If you are using Podman, simply replace `docker` with `podman` — or set the alias `alias docker=podman` once and forget about it.

1.4 Installing Docker or Podman

1.4.1 Docker

On Linux, install Docker Engine following the [official documentation](#). On Mac and Windows, Docker Desktop is the easiest option if your license allows it: download it from docs.docker.com/desktop.

1.4.2 Podman (Mac)

On macOS, Podman uses a lightweight Linux virtual machine behind the scenes — the same approach Docker Desktop uses. Install and initialise it with three commands:

```
brew install podman
podman machine init
podman machine start
```

After this, `podman` is ready and behaves exactly like `docker`.

1.5 Core Concepts: Images and Containers

Before running anything, two concepts are essential.

1.5.1 Docker Image

A **Docker image** is a read-only, layered package that contains everything needed to run a specific application: the OS filesystem, runtime, libraries, configuration files, and the application itself.

Images are stored in registries. The most popular is [Docker Hub](#), which hosts thousands of official and community images. When you run a container, Docker (or Podman) downloads the image from the registry if it is not already present on your machine.

Images are built in **layers**. Each instruction that modifies the filesystem adds a layer on top of the previous ones. Layers are cached and shared between images — this is what keeps images small and builds fast.

1.5.2 Docker Container

A **container** is a running instance of an image. You can think of the image as a class and the container as an object instantiated from it. Multiple containers can run from the same image simultaneously, each fully isolated from the others.

Containers share the host OS kernel but are isolated from each other using Linux kernel features: **namespaces** (for process, network, filesystem isolation) and **cgroups** (for resource limits like CPU and memory).

This is the fundamental difference from a Virtual Machine: a VM emulates an entire hardware stack and runs a separate OS kernel. A container shares the host kernel and is much lighter as a result.

We will explore this topic in depth in the [next article](#).

1.6 Essential Commands

Here is the minimal set of commands you need to start working with Docker. These are the ones you will use every day.

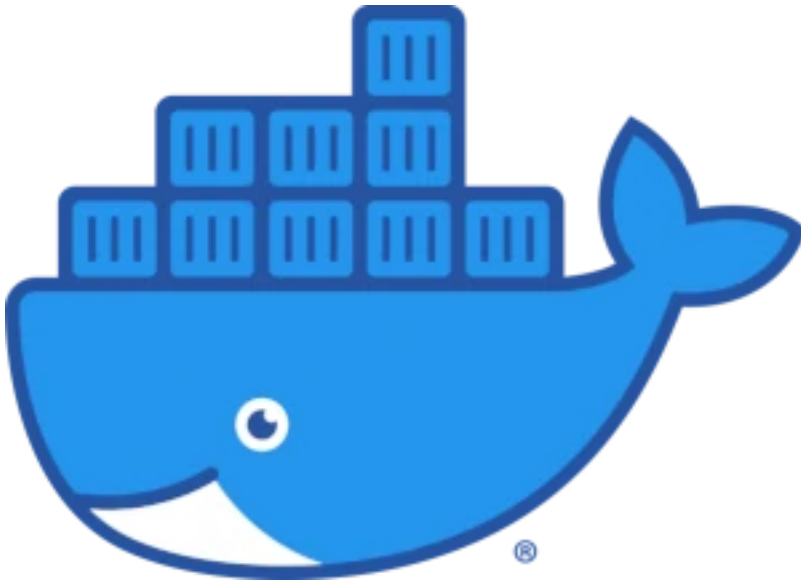


Figure 1.2: Container vs VM

1.6.1 List local images

```
docker image ls
```

At the beginning, the list is empty.

1.6.2 Pull an image from Docker Hub

```
docker pull nginx:alpine
```

This downloads the official Nginx image based on Alpine Linux — a very small (~40 MB) but fully functional web server image.

1.6.3 Run a container

```
docker run -d -p 8080:80 --name my-nginx nginx:alpine
```

Breaking down the options:

- `-d` — runs the container in detached mode (in the background, shell does not hang)
- `-p 8080:80` — maps port 8080 on your host to port 80 inside the container
- `--name my-nginx` — gives the container a friendly name instead of a random one
- `nginx:alpine` — the image to use

Open your browser at `http://localhost:8080` and you will see the Nginx welcome page. You just ran a web server without installing anything on your machine.

1.6.4 List running containers

```
docker container ls
```

To see all containers including stopped ones:

```
docker container ls -a
```

1.6.5 View container logs

```
docker logs my-nginx
```

1.6.6 Execute a command inside a running container

```
docker exec -it my-nginx /bin/sh
```

The `-it` flags attach an interactive terminal. Note: Alpine-based images use `sh` instead of `bash`.

1.6.7 Stop and remove a container

```
docker stop my-nginx
```

```
docker rm my-nginx
```

Or in one command:

```
docker rm -f my-nginx
```

1.6.8 Remove an image

```
docker image rm nginx:alpine
```

1.6.9 Clean up everything unused

```
docker system prune
```

This removes all stopped containers, dangling images, and unused networks. Add `-a` to also remove unused images that are not referenced by any container.

1.7 Your First Containerized Application

Let us put it all together with a practical example. You will serve a custom HTML page through Nginx — without installing Nginx, without a Dockerfile, using only the official image and a single `docker run` command.

Create a file called `index.html` anywhere on your machine:

```
<!DOCTYPE html>
<html>
  <head><title>Hello from Docker</title></head>
  <body>
    <h1>Hello, World!</h1>
    <p>This page is served by Nginx running inside a container.</p>
  </body>
</html>
```

Now run Nginx and mount that file into the container:

```
docker run -d \
  -p 8080:80 \
  --name hello-nginx \
  -v $(pwd)/index.html:/usr/share/nginx/html/index.html:ro \
  nginx:alpine
```

The `-v` option **bind mounts** your local file into the container's web root. The `:ro` suffix makes the mount read-only inside the container (a good practice).

Open `http://localhost:8080` in your browser:

Hello, World! This page is served by Nginx running inside a container.

You served a custom page without installing a web server, without writing a Dockerfile, and

without touching your system configuration. When you are done:

```
docker rm -f hello-nginx
```

Your machine is exactly as it was before. Nothing installed, nothing left behind.

1.8 How This Series Is Structured

This first article introduced the core concepts and got you running immediately. In the upcoming articles we will progressively build on this foundation:

- **Containers vs Virtual Machines** — deep dive into isolation mechanisms, namespaces, cgroups, and the difference from hypervisor-based virtualization
- **Dockerfile & Building Custom Images** — write your own Dockerfile, build and tag a custom image, understand the layer cache
- **How Docker Networking Works** — user-defined bridge networks, automatic DNS resolution between containers, network isolation
- **How Docker Volumes Work** — named volumes vs bind mounts, persisting data across container restarts
- **How Docker Compose Works** — manage multi-container applications with a single `docker compose up`
- **Docker Security Best Practices** — run containers as non-root, drop capabilities, scan images for vulnerabilities

Each article builds on the previous one around the same practical Nginx application, adding one concept at a time.

1.9 Conclusion

In this article we covered:

- The problem Docker solves and why containers matter
- The difference between Docker and Podman, and when to use each
- The two fundamental concepts: **images** and **containers**
- The essential CLI commands for daily use
- Running a custom web page with Nginx using only the official image

The [next article](#) goes deeper into the conceptual foundations: what actually makes a container different from a virtual machine, and what Linux kernel mechanisms make it all work.

Chapter 2

Containers vs Virtual Machines

2.1 Introduction

This is the second article of the **Getting Started with Docker** series. In the [previous article](#) we introduced Docker and ran our first container. Before going further, it is worth spending time on a fundamental question: **what is the real difference between a container and a virtual machine?**

When I started working with Docker this was not easy to grasp. The two technologies look similar from the outside — both give you an isolated environment to run software — but they work in completely different ways under the hood.

Virtualization can be achieved in two ways:

- **Full virtualization** — a hypervisor simulates enough hardware to run a complete guest operating system. This is the Virtual Machine approach.
- **Operating system-level virtualization** — the OS kernel itself is used to create isolated environments that share the same kernel. This is the Container approach.

2.2 Virtual Machines

Historically, as server processing power grew, applications running on bare metal couldn't exploit the abundance of available resources. **Virtual Machines** were born to solve this: by running a hypervisor on top of physical hardware, you could carve up one server into many independent guest systems.

A **hypervisor** (also called Virtual Machine Monitor) is the software layer that sits between the hardware and the virtual machines. It can be type 1 (bare metal, like VMware ESXi or KVM) or

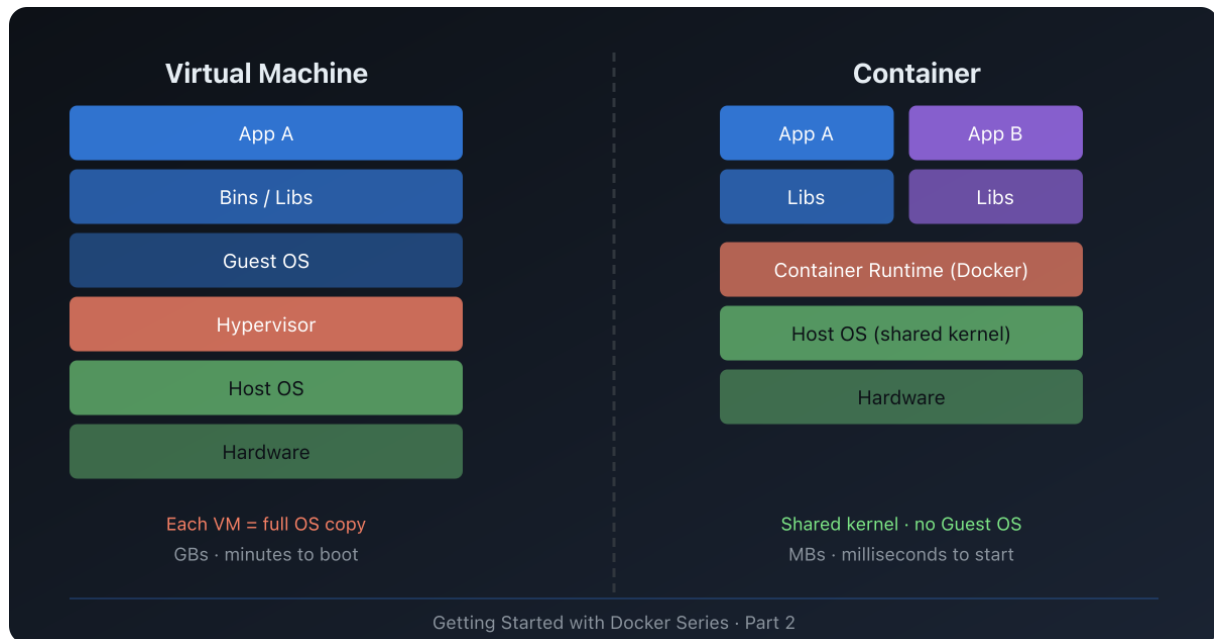


Figure 2.1: Containers vs Virtual Machines

type 2 (hosted, like VirtualBox or Parallels).

Each VM includes:

- A full copy of a guest operating system
- Virtual hardware (CPU, RAM, disk, network)
- All the application dependencies

The system running the VMs is called the **host**, the systems running inside are called **guests**.

2.2.1 Pros and cons of VMs

Pros

Strong isolation – each VM has its own kernel.

Run different operating systems on the same host (Windows on Linux, etc.).

Create and destroy on demand in minutes.

Snapshot a VM at any point in time.

Foundation of the IaaS cloud model (AWS EC2, Azure VMs, etc.).

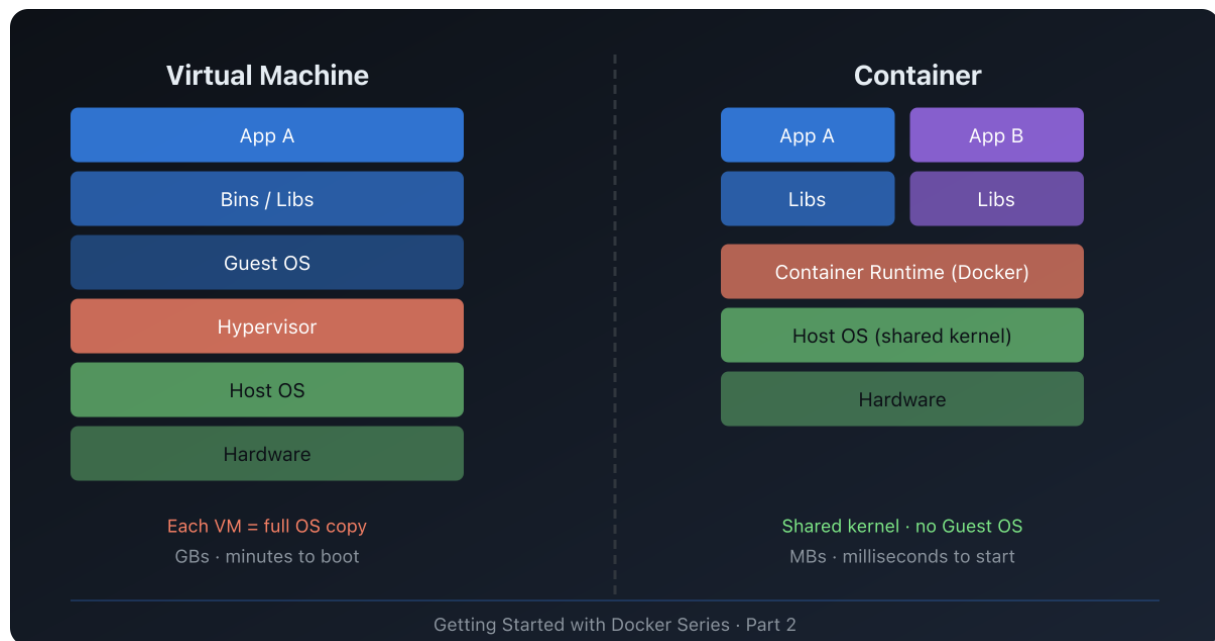


Figure 2.2: VM vs Container stack — VM has a full Guest OS per application, containers share the host kernel

Cons

Heavy — each VM carries a full OS (GBs of disk, hundreds of MB of RAM).
Slow to boot — minutes, not seconds.
Hypervisor adds overhead to application execution.
Managing a large fleet of VMs is operationally expensive.

The rise of VMs gave birth to the **Infrastructure as a Service (IaaS)** market. Companies like Amazon, Microsoft, and IBM allow you to rent virtual machines on demand. A startup today does not need to buy physical hardware — it registers an account with a cloud provider, orders a VM with the required specs, and is ready in minutes.

2.3 Containers

A **container** is an isolated environment created using features of the Linux kernel itself — not a separate hypervisor. Multiple containers run on the same host, all sharing the same OS kernel, but each seeing its own isolated filesystem, processes, network, and users.

The two kernel mechanisms that make this possible are:

- **Namespaces** — give each container an isolated view of the system: its own process tree (PID namespace), network interfaces (net namespace), filesystem (mount namespace), users (user namespace), and more.
- **cgroups** (control groups) — limit and account for resource usage (CPU, memory, disk I/O) per container.

Together, namespaces and cgroups create the illusion of a separate machine without the overhead of emulating hardware or running a second kernel.

Important note for Mac and Windows users: because containers rely on Linux kernel features, Docker and Podman on macOS and Windows run a lightweight Linux VM behind the scenes (using Apple's Virtualization Framework on Mac, and WSL2 on Windows). Your containers still run in that Linux environment — you just don't see it.

2.2.1 Pros and cons of containers

Pros

Lightweight — a container image is tens or hundreds of MB, not GBs.
Fast — containers start in milliseconds to seconds.
No overhead from emulating hardware.
Portable — same image runs on your laptop, CI server, and production.
Solves the "it works on my machine" problem.
Natural fit for microservices architecture.

Cons

Weaker isolation than VMs — all containers share the host kernel.
Cannot run a different OS kernel (e.g., no Windows containers on Linux).
Managing many containers requires an orchestrator (Kubernetes, Swarm).
Security requires care — a misconfigured container can affect the host.

2.4 The Analogy

Mike Coleman on the [Docker official blog](#) provides a memorable analogy:

Houses (VMs) are fully self-contained and offer protection from unwanted guests. They each possess their own infrastructure — plumbing, heating, electrical. Apartments (containers) also offer protection from unwanted guests, but they are built around shared infrastructure. The apartment building (Docker Host) shares plumbing, heating, electrical. Apartments are offered in all kinds of sizes — from studio to penthouse. You only rent exactly what you need.

2.5 Containers and VMs: not enemies, complementaries

In practice, containers and VMs are often used **together**, not in opposition. The most common production setup is:

1. A cloud provider gives you a VM (an EC2 instance, an Azure VM, etc.)
2. Docker or a container runtime runs inside that VM
3. Your application runs as containers inside Docker

VMs provide the strong isolation boundary at the infrastructure level. Containers provide the lightweight, fast, portable packaging for applications within that boundary.

2.6 Conclusion

In this article we covered:

- The difference between full virtualization (VMs) and OS-level virtualization (containers)
- How **namespaces** and **cgroups** implement container isolation
- Pros and cons of each technology
- Why containers and VMs are complementary, not competing

The [next article](#) focuses on **Dockerfiles**: how to write your own image definition and build a custom container image from scratch.

Chapter 3

Dockerfile and Building Custom Images

3.1 Introduction

This is the third article of the **Getting Started with Docker** series. In the [previous article](#) we explored the differences between containers and virtual machines. In the [first article](#) we ran Nginx directly from the official image using a bind mount to serve a custom page.

That approach works well for quick experiments, but in a real project you want a **reproducible, self-contained image** that already includes your content and configuration — one that anyone on your team (or any CI/CD pipeline) can build and run without manual steps.

This is where **Dockerfiles** come in.

3.2 What is a Dockerfile?

A **Dockerfile** is a plain-text file containing a sequence of instructions that tell Docker how to build an image. Each instruction creates a new **layer** on top of the previous one. When you rebuild an image, Docker only rebuilds the layers that changed — everything before the first change is served from the **layer cache**, making subsequent builds very fast.

3.3 Dockerfile Instructions

Here are the most important instructions you will use in every Dockerfile.

3.3.1 FROM

```
FROM nginx:alpine
```




Figure 3.1: Dockerfile and Building Custom Images

Every Dockerfile must start with `FROM`. It specifies the **base image** — the starting point. In this series we always start from `nginx:alpine`, a minimal image that already contains a working Nginx web server.

3.3.2 COPY

```
COPY ./html /usr/share/nginx/html
```

Copies files or directories from your build context (the folder where you run `docker build`) into the image. This is the main instruction for adding your application code or configuration.

3.3.3 ADD

`ADD` is similar to `COPY` but with two extra powers: it can automatically unpack compressed archives (`.tar.gz`, etc.) and it can fetch files from remote URLs. For simply copying local files, **prefer `COPY`** — it is explicit and predictable.

3.3.4 RUN

```
RUN apk add --no-cache curl
```

Executes a command **during the build** and commits the result as a new layer. Use it to install

packages, compile code, or run any setup step. Each RUN instruction is a layer, so chain related commands with && to minimize layer count.

3.3.5 ENV

```
ENV APP_ENV=production
```

Sets an environment variable that is available both during the build and at container runtime.

3.3.6 EXPOSE

```
EXPOSE 80
```

Documents which port the container listens on. It does **not** actually publish the port — that happens at runtime with -p. Think of it as metadata for the image user.

3.3.7 CMD

```
CMD ["nginx", "-g", "daemon off;"]
```

Specifies the default command to run when a container starts. It can be overridden at runtime. Use the **exec form** (JSON array) rather than the shell form to avoid wrapping the process in a shell — this ensures signals like SIGTERM are delivered directly to your process.

3.3.8 ENTRYPOINT

Similar to CMD but not easily overridden. Use ENTRYPOINT when the container has a fixed main executable, and CMD to provide default arguments to it.

3.3.9 USER

```
USER nginx
```

Switches to a non-root user for all subsequent instructions and at runtime. This is a **security best practice** — we will cover it in detail in the security article.

3.4 Building a Custom Nginx Image

Let us build a custom image that already includes our HTML page, so anyone can run it with a single `docker run` — no bind mounts, no extra files needed.

Create a project directory:

```
mkdir my-nginx && cd my-nginx
```

Create the HTML content:

```
mkdir html
cat > html/index.html << 'EOF'
<!DOCTYPE html>
<html>
  <head><title>My Custom Nginx</title></head>
  <body>
    <h1>Hello from my custom image!</h1>
    <p>This page is baked into the Docker image.</p>
  </body>
</html>
EOF
```

Create the Dockerfile:

```
FROM nginx:alpine

# Copy our HTML content into the image
COPY html /usr/share/nginx/html

# Document which port Nginx listens on
EXPOSE 80
```

Build the image:

```
docker build -t my-nginx:1.0 .
```

Breaking down the command:

- `-t my-nginx:1.0` — **tag** the image with a name and version
- `.` — the **build context** (current directory); Docker sends all files here to the build engine



Figure 3.2: Dockerfile instructions and image layers

List your images to confirm it was created:

```
docker image ls
```

Run a container from your custom image:

```
docker run -d -p 8080:80 --name my-app my-nginx:1.0
```

Open `http://localhost:8080` — you will see your custom page, served directly from the image without any bind mounts.

3.5 Understanding the Layer Cache

Run `docker build` a second time without changing anything:

```
docker build -t my-nginx:1.0 .
```

You will see output like:

```
=> CACHED [1/2] FROM docker.io/library/nginx:alpine
=> CACHED [2/2] COPY html /usr/share/nginx/html
```

Both steps are served from cache — the build is instant. Now edit `html/index.html` and rebuild:

```
docker build -t my-nginx:1.1 .
```

Only the COPY layer is rebuilt. The FROM layer (downloading the base image) is still cached.

Rule of thumb: order your Dockerfile instructions from least to most frequently changing. Put RUN commands that install packages near the top, and COPY of your application code near the bottom. This maximises cache reuse and keeps builds fast.

3.6 Tagging and Versioning Images

Good image tags tell you exactly what version is inside:

```
# Tag with a version number
docker build -t my-nginx:1.0 .

# Tag the same image as latest
docker tag my-nginx:1.0 my-nginx:latest
```

Always use explicit version tags in production. The latest tag is convenient locally but can cause surprises in automated pipelines because it changes silently.

3.7 Cleaning Up

Remove the running container:

```
docker rm -f my-app
```

Remove the image:

```
docker image rm my-nginx:1.0 my-nginx:1.1
```

Remove all unused images and build cache:

```
docker system prune -a
```

3.8 Conclusion

In this article we covered:

- The purpose of a **Dockerfile** and how it defines an image
- The most important instructions: FROM, COPY, RUN, ENV, EXPOSE, CMD, ENTRYPOINT, USER
- How to **build**, **tag**, and **run** a custom Nginx image
- How the **layer cache** works and how to structure a Dockerfile to maximise it

The [next article](#) introduces Docker networking: how to connect containers together so they can communicate over a private network.

Chapter 4

How Docker Networking Works

4.1 Introduction

This is the fourth article of the **Getting Started with Docker** series. In the [previous article](#) we built a custom Nginx image. So far, each container has been an isolated island. In a real application you often have multiple containers that need to talk to each other — a frontend container, a backend container, a database.

This is where Docker networking comes in.

4.2 Networking Overview in Docker

Docker has a pluggable networking system. The networking plugins are called **drivers**. Docker provides these drivers out of the box:

1. **bridge** — the default; creates a private internal network on the host
2. **host** — the container shares the host's network stack directly
3. **overlay** — connects containers running on different hosts (used in Swarm mode)
4. **macvlan** — assigns a real MAC address to a container, making it appear as a physical device on the network
5. **ipvlan** — similar to macvlan but operates at layer 3; useful when the upstream switch does not allow multiple MACs per port
6. **none** — networking is completely disabled

For this series we focus on **bridge**, which is what you will use for local development and single-host deployments.

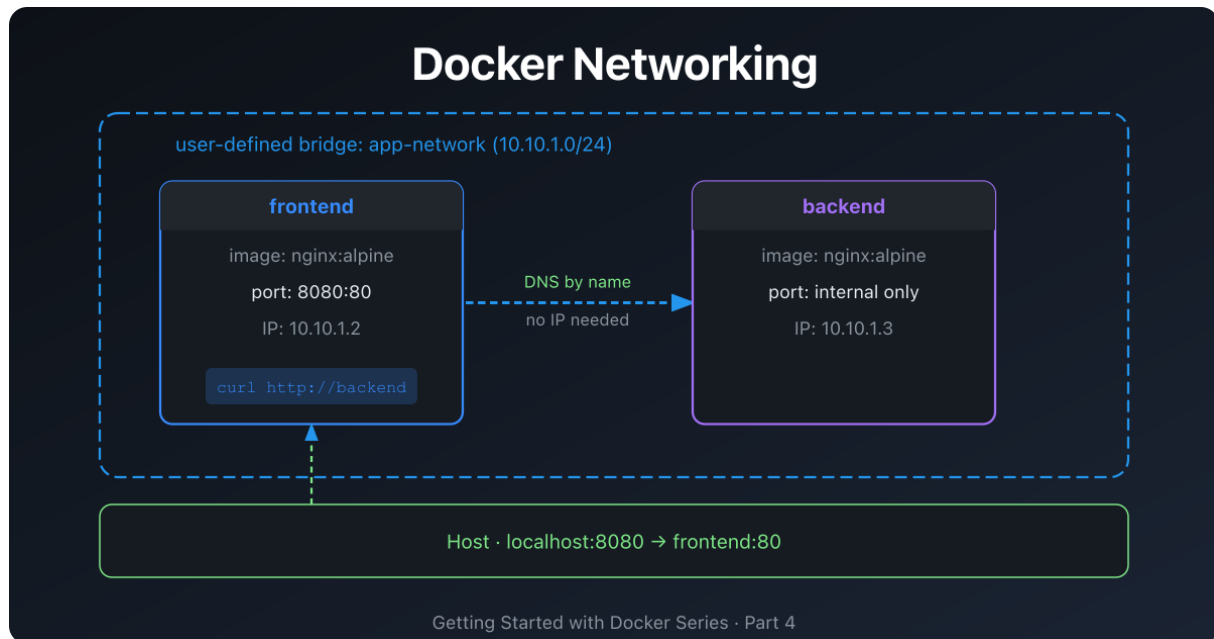


Figure 4.1: How Docker Networking Works

4.2.1 The bridge driver

The **bridge** driver creates a private network on the host. Containers attached to the same bridge can communicate with each other. External access is granted by publishing ports with `-p`.

Behind the scenes, Docker creates a Linux bridge interface, assigns IP addresses from a private subnet, and manages iptables rules to route traffic in and out.

There are two kinds of bridge networks:

- **default bridge** (bridge) — the one Docker uses automatically when you run a container without specifying a network. Containers can reach each other only by IP address, not by name.
- **user-defined bridge** — created with `docker network create`. Containers on the same user-defined bridge can **resolve each other by container name** via Docker's built-in DNS. This is the right choice for any multi-container setup.

4.2.2 The host driver

With the **host** driver the container skips Docker's network layer entirely and shares the host's network interfaces. Port mapping (`-p`) is not needed — if the container listens on port 80, it is directly reachable on the host's port 80.

Useful for performance-sensitive workloads or tools that need to monitor host network traffic. Note: `--network host` does not work on Docker Desktop for Mac or Windows — it only works natively on Linux.

4.2.3 The overlay driver

The **overlay** driver enables containers on different physical hosts to communicate as if they were on the same network. It is the networking backbone for **Docker Swarm** clusters. We will cover this when we discuss Swarm in a future article.

4.2.4 The macvlan driver

The **macvlan** driver assigns a MAC address to a container. This makes it appear as a physical device directly on the network segment. It is the right choice for legacy applications that expect to be directly connected to the physical network rather than routed through Docker's network stack.

4.3 Docker Networking Commands

Create a user-defined bridge network:

```
docker network create my-network
```

To specify a subnet and gateway explicitly:

```
docker network create \
  -d bridge \
  --subnet=10.10.1.0/24 \
  --gateway=10.10.1.1 \
  my-network
```

List all networks:

```
docker network ls
```

Inspect a network (shows connected containers, subnet, gateway):

```
docker network inspect my-network
```

Remove a network:

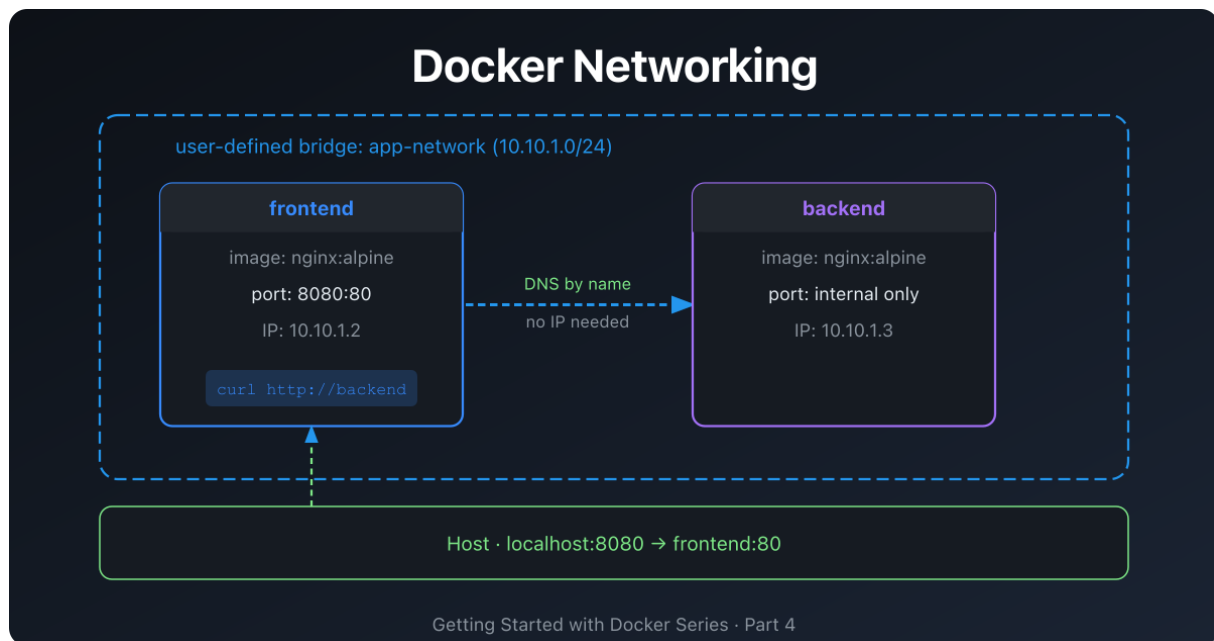


Figure 4.2: Docker networking — frontend and backend on the same bridge

```
docker network rm my-network
```

Connect a running container to a network:

```
docker network connect my-network my-container
```

4.4 Practical Example: Frontend and Backend on the Same Network

Let us build a concrete example. We will run two containers on the same user-defined bridge network and have them communicate by name — no IP addresses needed.

4.4.1 Create the network

```
docker network create app-network
```

4.4.2 Start the “backend” container

For simplicity we use another Nginx instance as a mock backend serving a JSON response.

```
mkdir backend
cat > backend/index.html << 'EOF'
{"status": "ok", "message": "Hello from the backend!"}
EOF

docker run -d \
  --name backend \
  --network app-network \
  -v $(pwd)/backend:/usr/share/nginx/html:ro \
  nginx:alpine
```

4.4.3 Start the “frontend” container

```
mkdir frontend
cat > frontend/index.html << 'EOF'
<!DOCTYPE html>
<html>
  <head><title>Frontend</title></head>
  <body>
    <h1>Frontend container</h1>
    <p>Backend is reachable at http://backend</p>
  </body>
</html>
EOF

docker run -d \
  --name frontend \
  --network app-network \
  -p 8080:80 \
  -v $(pwd)/frontend:/usr/share/nginx/html:ro \
  nginx:alpine
```

4.4.4 Verify containers can resolve each other by name

```
docker exec frontend ping -c 3 backend
```

Because both containers are on the same user-defined bridge network, Docker’s built-in DNS resolves backend to the correct container IP automatically. No need to hardcode IP addresses

anywhere.

Open `http://localhost:8080` in your browser to see the frontend. The backend is reachable internally at `http://backend` from within the frontend container.

4.4.5 Clean up

```
docker rm -f frontend backend
docker network rm app-network
```

4.5 Conclusion

In this article we covered:

- Docker's five network drivers: **bridge**, **host**, **overlay**, **macvlan**, **ipvlan**, and **none**
- The difference between the **default bridge** and **user-defined bridge** networks
- How **automatic DNS resolution** works on user-defined bridges (containers talk by name, not IP)
- Essential networking commands: `create`, `ls`, `inspect`, `rm`, `connect`
- A practical example with frontend and backend containers communicating by name

The [next article](#) covers **Docker volumes**: how to persist data outside a container so it survives restarts and replacements.

Chapter 5

How Docker Volumes Work

5.1 Introduction

This is the fifth article of the **Getting Started with Docker** series. In the [previous article](#) we connected containers over a private network. Now we need to address another fundamental question: **what happens to your data when a container is stopped or replaced?**

By default, a container's filesystem is **ephemeral** — everything written inside it disappears when the container is removed. This is fine for stateless services like a web server, but it is a problem as soon as you have anything that needs to survive: logs, configuration, user data, database files.

Docker solves this with **volumes**.

5.2 The Upgrade Problem

One of Docker's main benefits is that upgrading an application is as simple as replacing the old container with a new one. But if your data is stored inside the container, replacing it means losing the data.

The solution is to **separate the application binaries from the data**. Store the binaries in the image and the data in a volume that lives outside the container. When you replace the container, the volume stays where it is and the new container picks it up.

5.3 Docker Storage Types

Docker provides three options for persisting data from a container to the host filesystem.

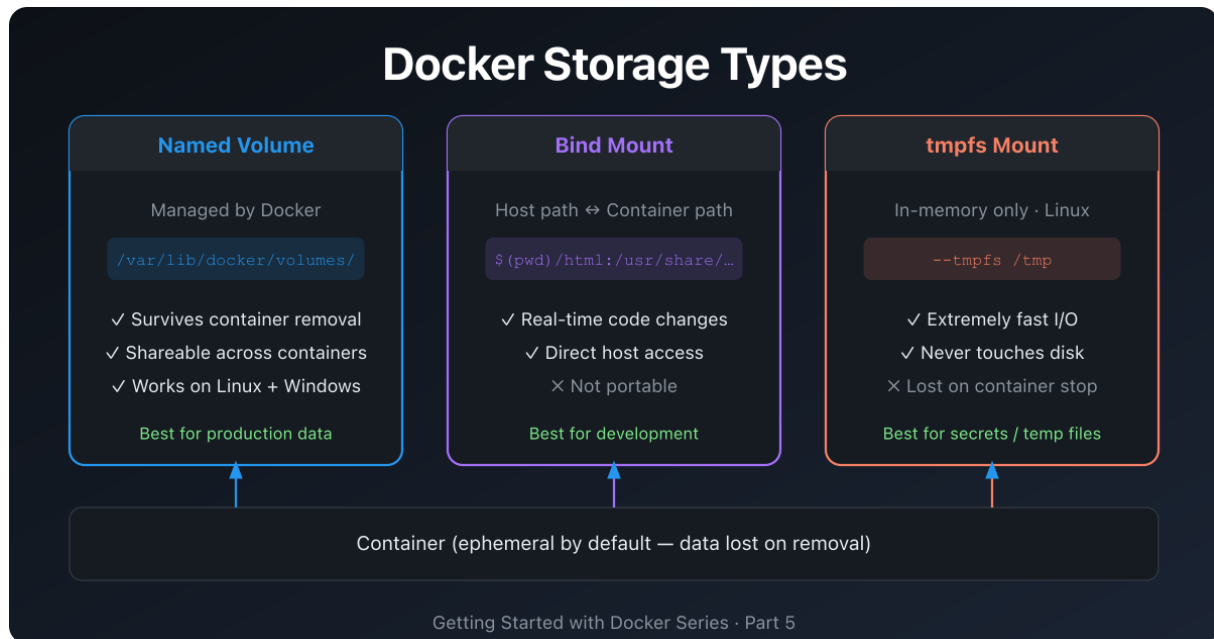


Figure 5.1: How Docker Volumes Work

5.3.1 Volumes

Volumes are managed entirely by Docker. They are stored in a directory on the host that Docker controls (typically `/var/lib/docker/volumes/` on Linux) and you interact with them through Docker CLI commands — not by poking around the filesystem directly.

Volumes are the **recommended way to persist data** in Docker. From the [official documentation](#), some of the reasons:

- Easier to back up and migrate than bind mounts
- Manageable through the Docker CLI and API
- Work on both Linux and Windows containers
- Can be safely shared among multiple containers
- Support remote storage drivers (NFS, cloud block storage, etc.)
- New volumes can be pre-populated with data from a container

5.3.2 Bind Mounts

Bind mounts map a specific directory or file on the **host filesystem** into the container. Unlike volumes, bind mounts are not managed by Docker — any process on the host can read and write them.

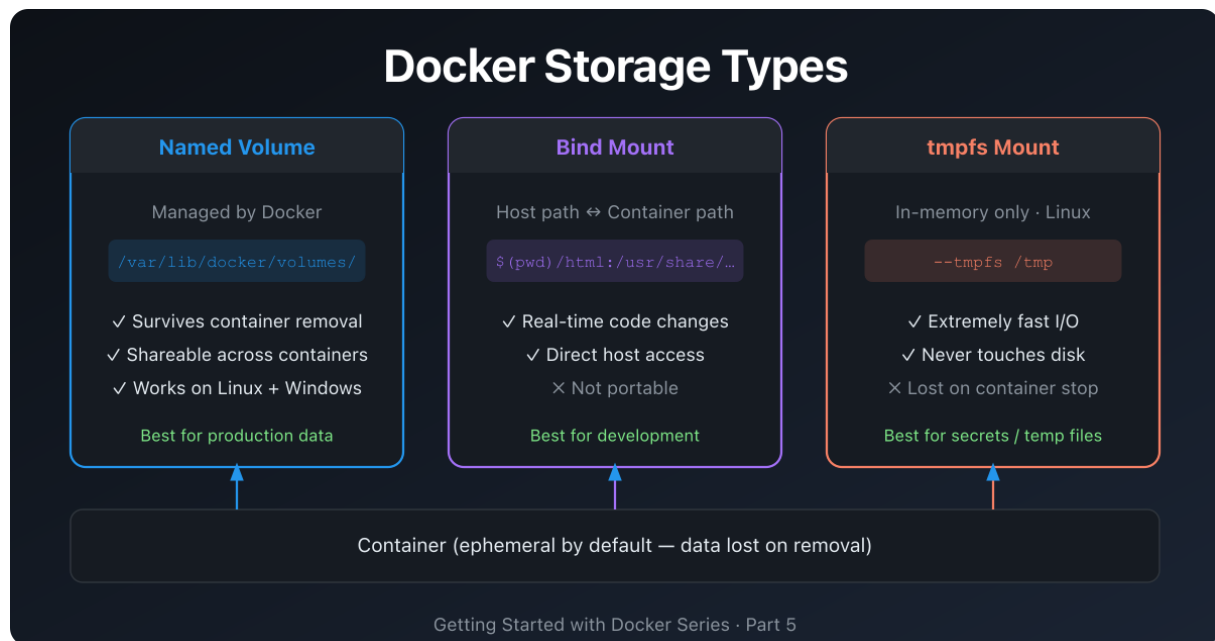


Figure 5.2: Docker storage types: volumes, bind mounts, tmpfs

Bind mounts are ideal when:

- You want to share source code between your editor on the host and the running container (live reload during development)
- You need the storage to be on a specific filesystem type
- You need direct access to the files from the host without going through Docker

We already used a bind mount in the first article: `-v $(pwd)/index.html:/usr/share/nginx/html/index.ht`

5.3.3 Tmpfs Mounts

Tmpfs mounts store data in the host's memory, not on disk. The data is never written to disk and disappears when the container stops. They are useful for sensitive data (secrets, tokens) that should never touch disk, or for temporary scratch space that requires very fast I/O.

Tmpfs mounts are only available on Linux hosts.

5.4 The `--mount` Syntax

Docker has two ways to attach storage to a container: the older `-v` flag and the newer `--mount` flag. The `--mount` syntax is more explicit and is the one recommended by the official documentation today.

Comparison:

```
# Old syntax (-v)
docker run -v my-volume:/app/data nginx:alpine

# New syntax (--mount) – equivalent
docker run --mount type=volume,source=my-volume,target=/app/data nginx:alpine
```

The `--mount` form accepts key-value pairs:

- `type` — volume, bind, or tmpfs
- `source` (or `src`) — volume name or host path (for bind mounts)
- `target` (or `dst`) — path inside the container
- `readonly` — makes the mount read-only

Both syntaxes work; `--mount` is preferred in scripts and documentation for clarity.

5.5 Docker Volumes Cheat Sheet

Create a named volume:

```
docker volume create my-data
```

List all volumes:

```
docker volume ls
```

Inspect a volume (shows the mount point on the host):

```
docker volume inspect my-data
```

Remove a volume:

```
docker volume rm my-data
```

Remove all unused volumes:

```
docker volume prune
```


5.6 Practical Example: Nginx with Persistent Content

Let us demonstrate volumes with our Nginx container. We will create a volume, write content into it from a temporary container, then serve that content with Nginx — and show that the content survives a container replacement.

5.6.1 Create the volume

```
docker volume create web-content
```

5.6.2 Populate the volume

```
docker run --rm \
  --mount type=volume,source=web-content,target=/data \
  alpine sh -c 'echo "<h1>Content from a volume</h1>" > /data/index.html'
```

This starts a temporary Alpine container, mounts the volume at /data, writes a file, and immediately exits and removes itself (--rm). The volume and its content persist.

5.6.3 Serve the content with Nginx

```
docker run -d \
  --name nginx-volume \
  -p 8080:80 \
  --mount type=volume,source=web-content,target=/usr/share/nginx/html \
  nginx:alpine
```

Open <http://localhost:8080> — you see the page from the volume.

5.6.4 Replace the container — data survives

```
docker rm -f nginx-volume

docker run -d \
  --name nginx-volume-new \
  -p 8080:80 \
  --mount type=volume,source=web-content,target=/usr/share/nginx/html \
```

```
nginx:alpine
```

Open `http://localhost:8080` again. The same page is still there. The volume outlived the container.

5.6.5 Clean up

```
docker rm -f nginx-volume-new
docker volume rm web-content
```

5.7 Volumes vs Bind Mounts: When to Use Which

	Volumes	Bind Mounts
Managed by Docker	Yes	No
Portable across machines	Yes (with volume drivers)	No (path must exist on host)
Best for production data	Yes	No
Best for dev code sharing	No	Yes
Access from host directly	Via <code>docker volume inspect</code>	Direct filesystem path
Works on Windows containers	Yes	Limited

5.8 Conclusion

In this article we covered:

- Why containers are ephemeral by default and why that matters
- The three Docker storage types: **volumes**, **bind mounts**, and **tmpfs**
- The modern **--mount** syntax vs the older **-v** flag
- Essential volume commands: `create`, `ls`, `inspect`, `rm`, `prune`
- A practical example showing that volume data survives container replacement
- A comparison table to decide when to use volumes vs bind mounts

The [next article](#) introduces **Docker Compose**: how to manage a multi-container application with a single YAML file instead of juggling multiple `docker run` commands.

Chapter 6

How Docker Compose Works

6.1 Introduction

This is the sixth article of the **Getting Started with Docker** series. In the [previous article](#) we learned how to persist data with volumes. So far, to run our application we have been juggling multiple `docker run` commands — one for the network, one for each container, one for each volume. This quickly becomes hard to manage and impossible to share reliably with a team.

Docker Compose solves this: you describe your entire application in a single YAML file and launch everything with one command.

6.2 What is Docker Compose?

Docker Compose is a tool for defining and running multi-container Docker applications. You describe your services, networks, and volumes in a `docker-compose.yml` file, and Docker Compose translates that into the correct sequence of `docker` commands.

Important: the original `docker-compose` command (v1, written in Python) was deprecated in May 2023 and is no longer maintained. The current tool is **docker compose** (v2, written in Go), which is a plugin built directly into the Docker CLI. Always use `docker compose` (with a space, not a hyphen). If you are on Podman, use `podman compose` which is compatible with the same YAML format.

You can think of `docker compose` as a wrapper that orchestrates all the `docker` commands you would otherwise run manually — but declaratively, from a single file.

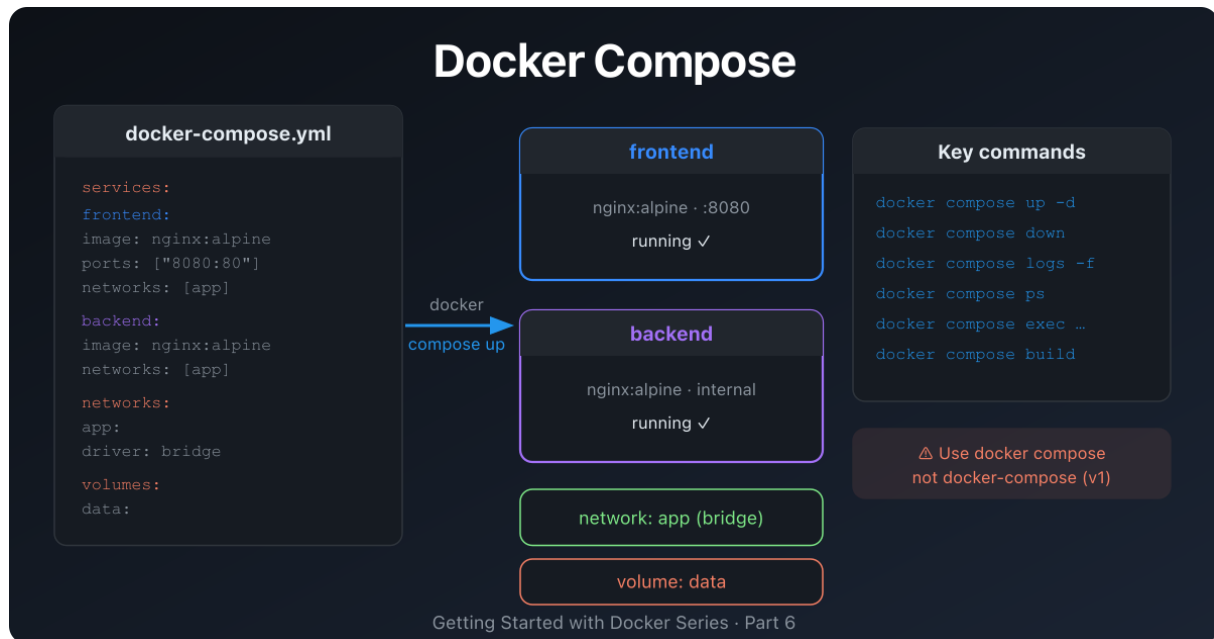


Figure 6.1: How Docker Compose Works

6.3 The docker-compose.yml File

The Compose file is a YAML document. Its top-level keys are:

- **services** — the containers that make up your application
- **volumes** — named volumes used by the services
- **networks** — custom networks (if not specified, Compose creates a default bridge network automatically)

6.3.1 A Note on the version Field

Older tutorials and documentation show a `version: '3.x'` field at the top of the Compose file. This field is **deprecated** and should be omitted. Modern Docker Compose uses the [Compose Specification](#) which does not require a version declaration.

6.4 Practical Example: Frontend and Backend with Compose

We will rebuild the two-container application from the networking article using Docker Compose, adding a named volume for the frontend content.

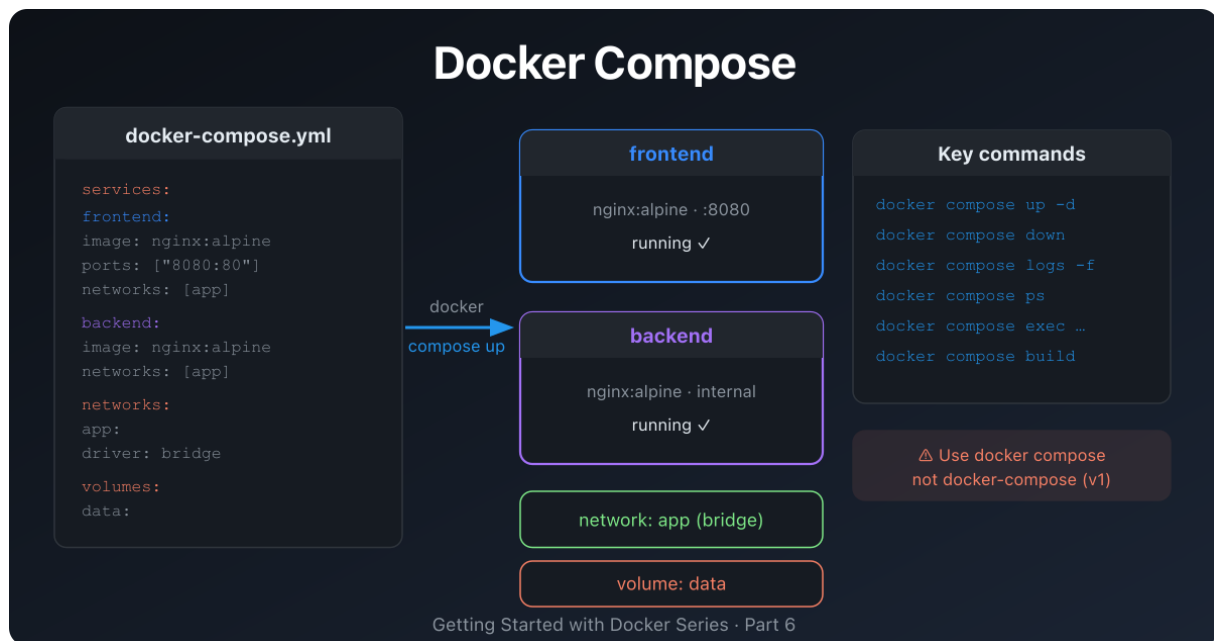


Figure 6.2: Docker Compose: YAML file maps to running containers, network, and volumes

6.4.1 Project structure

```

my-app/
├─ docker-compose.yml
├─ frontend/
│   └─ index.html
└─ backend/
    └─ index.html
  
```

Create the directories and content:

```

mkdir -p my-app/frontend my-app/backend
cd my-app

cat > frontend/index.html << 'EOF'
<!DOCTYPE html>
<html>
  <head><title>Frontend</title></head>
  <body>
    <h1>Frontend</h1>
    <p>This app is managed by Docker Compose.</p>
  </body>
</html>
  
```

```
EOF
```

```
cat > backend/index.html << 'EOF'
{"status": "ok", "message": "Hello from the backend!"}
EOF
```

6.4.2 The docker-compose.yml file

```
services:
  frontend:
    image: nginx:alpine
    container_name: frontend
    ports:
      - "8080:80"
    volumes:
      - ./frontend:/usr/share/nginx/html:ro
    networks:
      - app-network
    depends_on:
      - backend

  backend:
    image: nginx:alpine
    container_name: backend
    volumes:
      - ./backend:/usr/share/nginx/html:ro
    networks:
      - app-network

networks:
  app-network:
    driver: bridge
```

Key things to notice:

- **No version: field** — not needed with modern Compose
- **depends_on** — ensures the backend container starts before the frontend. For health-based ordering, you can use `condition: service_healthy` together with a healthcheck definition

- **Networks are created automatically** — Compose creates app-network for you
- **Bind mounts** use relative paths from the directory containing docker-compose.yml

6.5 Docker Compose Commands

The most important commands:

```
# Start all services (build images if needed, create networks and volumes)
docker compose up -d

# Stop and remove containers, networks (volumes are preserved)
docker compose down

# Stop and remove everything including volumes
docker compose down -v

# View logs of all services
docker compose logs

# Follow logs in real time
docker compose logs -f

# List running containers managed by this Compose file
docker compose ps

# Execute a command inside a running service container
docker compose exec frontend sh

# Rebuild images (useful after changing a Dockerfile)
docker compose build

# Pull the latest versions of all images
docker compose pull

# Restart a single service
docker compose restart frontend
```

The key difference from docker commands is that all of these act on the **entire application** defined in the Compose file, not a single container.

6.6 Environment Variables

Hard-coded values in `docker-compose.yml` are fine for development, but for anything sensitive (passwords, API keys) or environment-specific (URLs, port numbers) you should use variables.

Docker Compose automatically reads a file named `.env` in the same directory:

```
# .env
NGINX_PORT=8080
APP_ENV=development
```

Then reference them in `docker-compose.yml`:

```
services:
  frontend:
    ports:
      - "${NGINX_PORT}:80"
    environment:
      - APP_ENV=${APP_ENV}
```

Never commit `.env` files containing secrets to version control. Add `.env` to your `.gitignore`.

6.7 Starting the Application

From the `my-app/` directory:

```
docker compose up -d
```

Compose will:

1. Create the `app-network` bridge network
2. Start the backend container
3. Start the frontend container
4. Map port 8080 on your host to port 80 on the frontend container

Open `http://localhost:8080` — the frontend is running. Check that both services are up:

```
docker compose ps
```

Verify the backend is reachable by name from the frontend:


```
docker compose exec frontend ping -c 3 backend
```

6.7.1 Clean up

```
docker compose down
```

6.8 Conclusion

In this article we covered:

- What Docker Compose is and why it replaces manual `docker run` scripts
- The difference between `docker-compose v1` (deprecated) and **docker compose v2** (current)
- The structure of a `docker-compose.yml` file: services, volumes, networks
- Why the `version:` field is no longer needed
- Essential Compose commands: `up`, `down`, `logs`, `ps`, `exec`, `build`
- How to manage configuration with **environment variables** and `.env` files

The [next article](#) closes the series with **Docker security best practices**: how to run containers safely, limit their privileges, and scan images for vulnerabilities.

Chapter 7

Docker Security Best Practices

7.1 Introduction

This is the seventh article of the **Getting Started with Docker** series. In the [previous article](#) we orchestrated a multi-container application with Docker Compose. Now we turn our attention to **security**.

Containers are isolated — but isolation is not the same as security. A container running as root, with a writable filesystem and full Linux capabilities, is a significant risk if it is ever compromised. The good news is that Docker provides several mechanisms to reduce the attack surface, and most of them require only a few lines of configuration.

7.2 1. Never Run as Root

By default, processes inside a container run as **root** (UID 0). This means that if an attacker exploits a vulnerability in your application, they have root access inside the container — and depending on how the container is configured, they may be able to escape to the host.

The fix is simple: add a **USER** instruction to your Dockerfile.

```
FROM nginx:alpine

COPY html /usr/share/nginx/html

# Switch to the non-root user that the nginx image already provides
USER nginx

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```



Figure 7.1: Docker Security Best Practices



Figure 7.2: Docker security best practices overview

The official `nginx:alpine` image already includes a non-root `nginx` user. For your own images, create a dedicated user:

```
FROM alpine:3.20

RUN addgroup -S appgroup && adduser -S appuser -G appgroup

COPY --chown=appuser:appgroup app /app

USER appuser

CMD ["/app/start.sh"]
```

The `--chown` flag in `COPY` ensures the files are owned by the correct user from the start.

You can also enforce non-root at runtime without changing the Dockerfile:

```
docker run --user 1001:1001 nginx:alpine
```

7.3 2. Use a Read-Only Filesystem

If your application does not need to write to its own filesystem, make it read-only:

```
docker run --read-only -p 8080:80 nginx:alpine
```

If the application needs to write to specific paths (temp files, logs), mount those paths as `tmpfs`:

```
docker run --read-only \
  --tmpfs /tmp \
  --tmpfs /var/cache/nginx \
  --tmpfs /var/run \
  -p 8080:80 \
  nginx:alpine
```

A read-only filesystem means that even if an attacker gains code execution inside the container, they cannot modify the application binaries or install tools.

In Docker Compose:

```
services:
  frontend:
    image: nginx:alpine
    read_only: true
    tmpfs:
      - /tmp
      - /var/cache/nginx
      - /var/run
```

7.4 3. Drop Linux Capabilities

Linux capabilities are fine-grained divisions of root privileges. By default, Docker grants a container a subset of them. You can drop all capabilities and add back only what your application actually needs:

```
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE nginx:alpine
```

NET_BIND_SERVICE allows binding to ports below 1024 (like port 80). Most application containers do not need any other capability.

In Docker Compose:

```
services:
  frontend:
    image: nginx:alpine
    cap_drop:
      - ALL
    cap_add:
      - NET_BIND_SERVICE
```

7.5 4. Never Put Secrets in Environment Variables (or Images)

A common mistake is passing passwords or API keys through environment variables in `docker-compose.yml` or baking them into the image with ENV. Both approaches expose secrets in `docker inspect` output and in image layers.

What to do instead:

- Use **Docker secrets** (available in Swarm mode) which mount secrets as files in `/run/secrets/`

- Use a **secrets manager** (HashiCorp Vault, AWS Secrets Manager, etc.)
- As a minimum, use a `.env` file that is **never committed to version control** and reference it in Compose

```
# docker-compose.yml
services:
  app:
    image: my-app
    env_file:
      - .env.secrets  # never committed to git
```

Add to `.gitignore`:

```
.env
.env.secrets
*.env
```

Keep in mind what `env_file` actually protects against: it keeps the secret out of your Dockerfile and out of git. It does **not** keep the secret out of `docker inspect` — whatever ends up as a container environment variable, whether set with `-e`, `environment:`, or `env_file:`, is resolved into a real env var at container creation time and shows up in cleartext under `Config.Env` in `docker inspect`. Anyone with access to the Docker API or socket sees it — no shell access to the container required. Only file-based secrets (Docker secrets, a mounted volume, a secrets-manager agent) avoid that particular exposure, because they are never turned into an environment variable in the first place.

7.6 5. Use Minimal Base Images

Every package in your base image is a potential attack surface. Prefer minimal base images:

- `alpine` — ~5MB, very popular, uses musl libc
- `distroless` (Google) — no shell, no package manager, only the runtime and your app
- `scratch` — completely empty, for statically compiled binaries (Go, Rust)

For our Nginx example we already use `nginx:alpine`. Going even smaller would mean building a custom Nginx binary, which is rarely necessary.

The rule: **if it is not needed, it should not be in the image.**

7.7 6. Scan Images for Known Vulnerabilities

Even minimal images can contain packages with known CVEs. Docker provides a built-in scanner called **Docker Scout**:

```
# Quick vulnerability overview
docker scout quickview nginx:alpine

# Full CVE list
docker scout cves nginx:alpine

# Recommendations to fix vulnerabilities
docker scout recommendations nginx:alpine
```

Docker Scout is available in Docker Desktop and as a CLI plugin. It integrates with Docker Hub and can be added to CI/CD pipelines to fail a build when high-severity vulnerabilities are detected.

An alternative open-source scanner is **Trivy**:

```
trivy image nginx:alpine
```

Make image scanning part of your build pipeline, not an afterthought.

7.8 7. Keep Images Up to Date

A scanned image with no vulnerabilities today may have CVEs tomorrow. Pin your base image to a specific digest rather than a floating tag, and automate rebuild checks:

```
# Pin to a specific digest (reproducible, auditable)
FROM nginx:alpine@sha256:...
```

Many teams use tools like **Renovate** or **Dependabot** to automatically open PRs when a newer base image is available.

Pinning to a digest does not mean your image can never be updated — it means updates stop happening silently and start happening through your normal review process. The two ideas work together like this:

- **Tags are mutable, digests are not.** `nginx:alpine` is just a pointer that the Nginx team

can repoint to a new build at any time (for example, to patch a CVE). Today it might resolve to `sha256:aaa...`; next week the same tag could resolve to `sha256:bbb...` — different bytes, same tag. Pinning to `sha256:aaa...` gives you a byte-for-byte reproducible build, wherever and whenever it runs.

- **Even a fully specific version tag is still mutable.** It is tempting to think that a precise tag like `nginx:1.27-alpine3.20` is “pinned enough” because it names both the Nginx version and the Alpine version. It is not: that tag only fixes the *label*, not the *contents*. Alpine 3.20 is a supported release that keeps receiving security patches to its own packages (musl, OpenSSL, zlib, PCRE2...) for as long as it is maintained. When the Nginx maintainers rebuild the `1.27-alpine3.20` tag to pick up those patches, the tag name stays identical but the digest underneath it changes. You can see this yourself:

```
docker pull nginx:1.27-alpine3.20
docker inspect nginx:1.27-alpine3.20 --format ''
```

Run the same two commands again in a few weeks and there is no guarantee you get the same digest back. This is true of any tag on any registry (Docker Hub, GHCR, ECR...) — a tag is just a “name-to-digest” entry that anyone with push rights can reassign, and most registries do not lock it by default. The version in the tag narrows *how much* can change under your feet (you will not silently jump from 1.27 to 1.29), but only the digest gives you a cryptographic guarantee that the bytes never change.

- **Renovate/Dependabot poll the registry, not your running containers.** On a schedule (typically daily), the bot asks the registry what `nginx:alpine` currently resolves to, and compares it against the digest pinned in your Dockerfile. If they differ, it opens a pull request that changes just that one line — `FROM nginx:alpine@sha256:aaa...` becomes `FROM nginx:alpine@sha256:bbb...`. Nothing in production changes yet.
- **The PR goes through your usual pipeline.** CI builds the image at the new digest, runs your tests, and can re-run a vulnerability scan against it before anyone merges. If the new base image breaks something, you find out in the PR, not in production.
- **Only after merge** does your normal CI/CD build and deploy the image at the new, now-pinned digest.

In other words, a floating tag can silently hand you a different image on every build with no diff to review; a pinned digest plus an update bot turns every base image change into a reviewed, tested commit — you keep the ability to stay current without giving up reproducibility.

7.9 Putting It All Together

Here is a Dockerfile for our Nginx image that applies all the practices above:

```
FROM nginx:alpine

# Copy content
COPY --chown=nginx:nginx html /usr/share/nginx/html

# Drop to non-root
USER nginx

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

And the corresponding docker-compose.yml:

```
services:
  frontend:
    build: .
    image: my-nginx:secure
    container_name: frontend
    ports:
      - "8080:80"
    read_only: true
    tmpfs:
      - /tmp
      - /var/cache/nginx
      - /var/run
    cap_drop:
      - ALL
    cap_add:
      - NET_BIND_SERVICE
    networks:
      - app-network

networks:
  app-network:
    driver: bridge
```

7.10 Conclusion

In this article we covered:

- Running containers as a **non-root user** with USER in the Dockerfile
- Using a **read-only filesystem** with `--read-only` and `tmpfs` for writable paths
- **Dropping Linux capabilities** with `--cap-drop=ALL` and adding back only what is needed
- Keeping **secrets out of images and environment variables**
- Choosing **minimal base images** to reduce attack surface
- **Scanning images** for vulnerabilities with Docker Scout or Trivy
- **Keeping images up to date** with pinned digests and automated updates

The [next and final article](#) of the series ties everything together into a single checklist for writing Dockerfiles that are functional, lightweight, debuggable, and secure.

Chapter 8

Docker Image Best Practices

8.1 Introduction

This is the eighth and final article of the **Getting Started with Docker** series. In the [previous article](#) we hardened a container against attacks. Back in [Part 3](#) we wrote our first Dockerfile and learned the basic instructions and the layer cache.

Now it is time to put everything together. A Dockerfile that just “works” is not enough — a well-written image should be:

- **Functional** — it starts correctly, exposes the right ports, and tells Docker how to check its own health
- **Lightweight** — it ships only what the application needs to run, nothing else
- **Debuggable** — when something goes wrong at 2 AM, you can find out why quickly
- **Secure** — it does not run as root and does not carry unnecessary attack surface

This article collects the practices that matter most for each of these four dimensions, using the Nginx example from the series as a running example.

8.2 1. Functional: Make the Image Behave Predictably

8.2.1 Use the exec form for CMD and ENTRYPOINT

```
# Good — exec form, PID 1, signals delivered directly
CMD ["nginx", "-g", "daemon off;"]

# Avoid — shell form wraps the process, SIGTERM may not reach it
CMD nginx -g daemon off;
```



Figure 8.1: Docker Image Best Practices

The exec form (JSON array) runs your process as PID 1 directly, so `docker stop` can deliver SIGTERM to it and your application can shut down gracefully instead of being killed after the timeout.

8.2.2 Combine ENTRYPOINT and CMD

Use ENTRYPOINT for the fixed executable and CMD for default arguments the user can override:

```
ENTRYPOINT ["nginx"]
CMD ["-g", "daemon off;"]
```

Now `docker run my-nginx -v` runs `nginx -v` instead of the default arguments, without anyone needing to know the entrypoint.

8.2.3 Declare a HEALTHCHECK

```
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD curl -f http://localhost/ || exit 1
```

Without a HEALTHCHECK, Docker only knows whether the main process is alive — not whether your application is actually serving requests. With it, `docker ps` shows `healthy / unhealthy`,

and orchestrators (Compose, Swarm, Kubernetes) can restart or stop routing traffic to a broken container automatically.

8.2.4 Use ENV for configuration, not for secrets

```
ENV NGINX_PORT=80
```

Environment variables are the standard way to make an image configurable at runtime. Give sensible defaults in the Dockerfile and let users override them with `docker run -e` or Compose's `environment:` — but keep this to configuration, never credentials (see the [security article](#) for why).

8.2.5 Add a .dockerignore

```
.git
node_modules
*.md
Dockerfile
.env
```

A `.dockerignore` file works exactly like `.gitignore`: it keeps unnecessary or sensitive files out of the **build context**, which makes builds faster and prevents accidentally COPY-ing things like `.git` history or local `.env` files into the image.

8.3 2. Lightweight: Ship Only What You Need

8.3.1 Start from a minimal base image

```
FROM nginx:alpine
```

alpine-based images are a fraction of the size of their Debian/Ubuntu-based equivalents because they use `musl libc` and `busybox` instead of a full userland. For runtimes that support it, `distroless` images go even further — no shell, no package manager, just the runtime and your application.

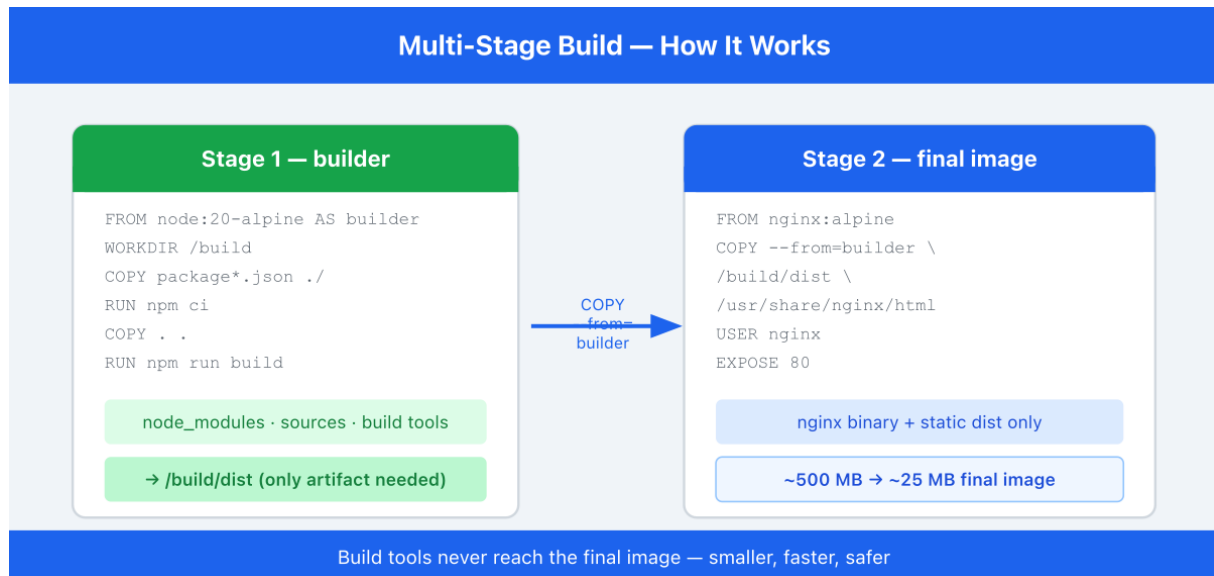


Figure 8.2: Multi-stage build diagram

8.3.2 Use multi-stage builds

This is the single most effective technique for keeping images small. Build tools, source code, and dependencies stay in an intermediate **builder stage** that never ships — only the final artifact is copied into the runtime image.

```
# Stage 1: build the static assets
FROM node:20-alpine AS builder
WORKDIR /build
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: serve them from a minimal image
FROM nginx:alpine
COPY --from=builder /build/dist /usr/share/nginx/html
USER nginx
EXPOSE 80
```

The `node_modules` folder, the TypeScript compiler, and every build dependency exist only in the builder stage. The final image contains just `nginx` and the compiled static files — often a 20x reduction in size, and a much smaller attack surface as a bonus.

When to reach for it: any time your build step needs tools your runtime does not — a compiler

or SDK (Go, Rust, Java/Maven, C/C++), a bundler/transpiler for a front-end app (webpack, vite, tsc), or a test suite you want to run during the build but never ship. As a rule, if `docker build` needs to install something with `apt/apk/npm/pip` just to *produce* an artifact, and the running application never calls that tool again, it belongs in a builder stage. It is also useful to keep build-time secrets (private registry tokens, `npm auth`) out of the final image's layer history, since the builder stage is simply discarded.

When it is not worth it: if your image never compiles or bundles anything — say, a Python script that only needs its runtime dependencies installed once, or a set of static files that already exist in the repository — a single stage is simpler and there is no build-only tooling to strip out. Multi-stage solves a specific problem (build tools leaking into the runtime image); do not add a stage just because it looks more sophisticated.

8.3.3 Chain RUN instructions and clean up in the same layer

```
# Good — one layer, no leftover cache
RUN apk add --no-cache curl \
    && curl -O https://example.com/tool.tar.gz \
    && tar -xzf tool.tar.gz \
    && rm tool.tar.gz

# Avoid — the download persists in an earlier layer even if a later RUN removes it
RUN apk add curl
RUN curl -O https://example.com/tool.tar.gz
RUN tar -xzf tool.tar.gz
RUN rm tool.tar.gz
```

Each `RUN` is a layer, and layers are additive — deleting a file in a later layer does not shrink the image, it only hides the file. Install, use, and clean up temporary files within the **same** `RUN` instruction. The `--no-cache` flag on `apk add` (or `rm -rf /var/lib/apt/lists/*` on Debian-based images) avoids leaving the package index cached in a layer.

8.3.4 Beware of `chown -R` in its own layer

A subtler version of the same problem: changing ownership of a large tree **after** it has already been copied.

```
# Avoid – chown touches every file, duplicating them into a new layer
COPY app /app
RUN chown -R appuser:appgroup /app

# Good – ownership is set as part of the copy, no extra layer
COPY --chown=appuser:appgroup app /app
```

chown looks like it only changes metadata, but the overlay filesystem Docker uses has to copy each affected file into the new layer to record that change. Run `RUN chown -R` over a directory that a previous layer already populated, and you can end up shipping that directory's contents **twice** — once in the `COPY` layer, once again in the `RUN chown` layer — roughly doubling its footprint in the image.

You can catch this with `docker history`, which lists the size of every layer in build order:

```
docker history my-nginx:1.2.0
```

If a `RUN chown -R ...` layer is anywhere near the size of the `COPY` layer above it, that is the tell — a metadata-only change should be close to 0 B. For a more visual, layer-by-layer breakdown (including a wasted-space estimate), the `dive` tool is built exactly for this:

```
dive my-nginx:1.2.0
```

The fix is almost always the same: set ownership with `--chown` on the `COPY` (or `ADD`) instruction that creates the files, instead of a separate `RUN chown -R` afterwards.

8.3.5 Order instructions from least to most frequently changing

As covered in [Part 3](#), put dependency installation before `COPY`-ing application code. Dependencies change rarely, code changes often — this ordering keeps the layer cache useful across builds.

8.4 3. Debuggable: Make Failures Easy to Diagnose

A lightweight, locked-down image is worth little if nobody can figure out why it crashed. A few practices keep images small and secure **without** sacrificing debuggability.

8.4.1 Log to stdout/stderr, not to files

```
# Nginx already does this – logs are symlinked to stdout/stderr
RUN ln -sf /dev/stdout /var/log/nginx/access.log \
    && ln -sf /dev/stderr /var/log/nginx/error.log
```

Docker captures anything written to stdout/stderr and makes it available through docker logs, which works with every logging driver and log aggregator. Logs written to a file inside the container are invisible unless you exec in and go looking for them.

```
docker logs -f my-app
```

8.4.2 Keep a shell available, or know how to add one temporarily

Distroless and scratch images are excellent for production but have no shell, so `docker exec -it my-app sh` will not work. If you need that debugging path, alpine is usually the right trade-off — it is still tiny but keeps `sh`, `curl`, and basic tools available:

```
docker exec -it my-app sh
```

If you are committed to a distroless final stage, keep a debug-friendly variant of the same image (many distroless images ship a `:debug` tag with Busybox included) for troubleshooting.

8.4.3 Add LABEL metadata

```
LABEL org.opencontainers.image.source="https://github.com/you/my-nginx" \
      org.opencontainers.image.version="1.0" \
      org.opencontainers.image.description="Custom Nginx image with static site"
```

When an incident happens at 2 AM, `docker inspect` on a labeled image immediately tells you which commit and which repository it was built from — no guessing which tag maps to which source.

8.4.4 Always use explicit, meaningful tags

Reusing `latest` makes it impossible to know which build is actually running. Tag every image with a version (and ideally the commit SHA), as shown in [Part 3](#):

```
docker build -t my-nginx:1.2.0 -t my-nginx:$(git rev-parse --short HEAD) .
```

8.5 4. Secure: Don't Undo Your Own Work

We covered security in depth in the [previous article](#); here is the short version to keep in mind while writing every Dockerfile:

- **Never run as root** — add a USER instruction, or run with `--user`
- **Prefer a read-only filesystem** at runtime, with `tmpfs` for the few paths that need writing
- **Drop Linux capabilities** with `--cap-drop=ALL` and add back only what is required
- **Never bake secrets** into ENV or image layers — use secrets managers or untracked `.env` files
- **Scan images** for known CVEs with Docker Scout or Trivy before shipping
- **Pin base images to a digest** and keep them updated

A lightweight image built with multi-stage builds is already a security win: fewer packages means fewer CVEs to track.

8.6 Putting It All Together

Here is the Nginx example from the series, now applying all four dimensions at once:

```
# — Stage 1: build —————
FROM node:20-alpine AS builder
WORKDIR /build
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# — Stage 2: runtime —————
FROM nginx:alpine

LABEL org.opencontainers.image.source="https://github.com/you/my-nginx" \
      org.opencontainers.image.version="1.2.0"
```

```
# Functional: only the compiled static assets, owned by the runtime user
COPY --from=builder --chown=nginx:nginx /build/dist /usr/share/nginx/html

# Debuggable: send Nginx logs to stdout/stderr
RUN ln -sf /dev/stdout /var/log/nginx/access.log \
    && ln -sf /dev/stderr /var/log/nginx/error.log

# Secure: drop to the non-root user the base image already provides
USER nginx

EXPOSE 80

# Functional: let Docker (and orchestrators) know when we're actually serving traffic
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
    CMD curl -f http://localhost/ || exit 1

ENTRYPOINT ["nginx"]
CMD ["-g", "daemon off;"]
```

Run it the same secure way we did in the previous article:

```
docker build -t my-nginx:1.2.0 .

docker run -d -p 8080:80 \
    --read-only \
    --tmpfs /tmp --tmpfs /var/cache/nginx --tmpfs /var/run \
    --cap-drop=ALL --cap-add=NET_BIND_SERVICE \
    --name my-app \
    my-nginx:1.2.0
```

8.7 Checklist

A quick reference to keep next to your Dockerfile:

- ☐ Exec form for CMD/ENTRYPOINT
- ☐ HEALTHCHECK defined
- ☐ .dockerignore excludes build artifacts, VCS metadata, and secrets
- ☐ Minimal base image (alpine, distroless, or scratch)

- ☐ Multi-stage build separates build tools from the runtime image
- ☐ Related RUN commands chained and cleaned up in the same layer
- ☐ Ownership set with COPY --chown rather than a separate RUN chown -R
- ☐ Instructions ordered from least to most frequently changing
- ☐ Application logs to stdout/stderr
- ☐ Image carries LABEL metadata and an explicit version tag
- ☐ USER set to a non-root user
- ☐ Runs with a read-only filesystem and dropped capabilities
- ☐ No secrets in ENV or image layers
- ☐ Scanned for CVEs before shipping

8.8 Conclusion

In this article we brought together everything the series covered, organized around four goals for every image you write:

- **Functional:** exec-form commands, HEALTHCHECK, sensible ENV defaults, and a `.dockerignore`
- **Lightweight:** minimal base images and **multi-stage builds** to keep build tools out of the final image
- **Debuggable:** logs to stdout/stderr, LABEL metadata, explicit tags, and a shell available when you need one
- **Secure:** non-root users, read-only filesystems, dropped capabilities, and vulnerability scanning

This concludes the **Getting Started with Docker** series. You now have a solid foundation to build, network, persist, orchestrate, secure, and package containerized applications the right way. The natural next step is **Docker Swarm** for multi-host deployments, or **Kubernetes** for large-scale container orchestration.

Glossary

Bind Mount A Docker storage mechanism that maps a specific file or directory on the host filesystem directly into a container. Unlike named volumes, bind mounts are not managed by Docker and depend on the host's directory structure.

BuildKit The modern build engine for Docker, enabled by default since Docker 23.0. BuildKit provides faster builds, better caching, and new features such as cache mounts and secret mounts in Dockerfiles.

cgroups (Control Groups) A Linux kernel feature that limits, accounts for, and isolates the resource usage (CPU, memory, disk I/O, network) of a collection of processes. Docker uses cgroups to enforce per-container resource limits.

Container A runnable instance of a Docker image. Containers are isolated from each other and from the host using Linux namespaces and cgroups, but share the host OS kernel.

containerd The industry-standard container runtime that Docker Engine uses under the hood since Docker 1.11. containerd manages the complete container lifecycle: image transfer, storage, execution, and supervision.

Docker Compose A tool for defining and running multi-container Docker applications using a declarative YAML file (`docker-compose.yml`). The current version is `docker compose` (v2, a Go plugin); the older `docker-compose` (v1, Python) was deprecated in May 2023.

Docker Desktop A GUI application for Mac and Windows that bundles Docker Engine, Docker CLI, Docker Compose, and Kubernetes. Requires a paid subscription for enterprise use.

Docker Hub The default public registry for Docker images. Hosts official images maintained by Docker and thousands of community images.

Docker Scout A vulnerability scanning tool integrated into the Docker CLI and Docker Desktop. Analyses images for known CVEs and provides remediation recommendations.

Dockerfile A plain-text file containing a sequence of instructions that tell Docker how to build an image. Each instruction creates a new layer in the image.

Image A read-only, layered template used to create containers. Images are built from Dockerfiles and stored in registries. Each layer represents a filesystem change.

Layer Cache The mechanism by which Docker reuses unchanged layers from previous builds. Layers are rebuilt only when an instruction or its inputs change, making subsequent builds significantly faster.

Namespace A Linux kernel feature that provides isolation for system resources. Docker uses multiple namespaces per container: PID (processes), NET (network interfaces), MNT (filesystem mounts), UTS (hostname), IPC, and USER.

OCI (Open Container Initiative) A set of open industry standards for container formats and runtimes. Both Docker and Podman produce OCI-compliant images, which means images built with one tool can be run by the other.

Podman A daemonless, rootless container engine that is fully compatible with the Docker CLI. An open-source alternative to Docker Desktop, particularly useful on corporate machines where Docker Desktop requires a commercial licence.

Registry A service that stores and distributes Docker images. Docker Hub is the default public registry. Private registries include AWS ECR, GitHub Container Registry, and self-hosted solutions.

Reverse Proxy A server that sits in front of backend services and forwards client requests to them. Nginx is commonly used as a reverse proxy in containerised applications.

tmpfs Mount A Docker storage mechanism that stores data in the host's RAM. The data is never written to disk and is lost when the container stops. Useful for sensitive data and high-speed temporary storage.

Volume A Docker-managed storage unit that persists data outside the container's filesystem. Volumes survive container removal and can be shared across multiple containers.

WSL2 (Windows Subsystem for Linux 2) A compatibility layer in Windows 10/11 that runs a real Linux kernel inside a lightweight VM. Docker Desktop uses WSL2 as its Linux backend on Windows.

What's Next?

You have reached the end of this book. If you have worked through each chapter, you now know how to:

- Pull and run containers from Docker Hub
- Explain the difference between containers and virtual machines at a kernel level
- Write Dockerfiles, build custom images, and understand the layer cache
- Create user-defined bridge networks and let containers resolve each other by name
- Persist data with named volumes and bind mounts
- Orchestrate a multi-container application with Docker Compose
- Harden a container image with security best practices

That is a complete, practical foundation. Most day-to-day Docker work does not require anything beyond what you have learned here.

Where to Go From Here

Docker Swarm

Docker Swarm is Docker's built-in clustering and orchestration mode. It lets you deploy multi-container applications across multiple hosts with a single `docker stack deploy` command. If you need high availability without the complexity of Kubernetes, Swarm is the right next step.

Kubernetes

Kubernetes (K8s) is the industry-standard container orchestrator for large-scale production deployments. It builds on the same container concepts you now know and adds automated roll-outs, self-healing, horizontal scaling, and much more. The [Getting Started with Kubernetes](#) series on Code4Projects is a natural continuation.

CI/CD with Docker and GitHub Actions

Automating the build, test, and push of your images in a CI/CD pipeline is the next productivity leap. GitHub Actions has first-class Docker support and can build multi-platform images (arm64 + amd64) with a single workflow file.

Docker Security (deeper)

This book covered the essentials. For production workloads, the next steps are: image signing with Docker Content Trust, runtime security with Falco, network policies, and secrets management with HashiCorp Vault or AWS Secrets Manager.

Stay in Touch

If this book helped you, consider:

- Subscribing to the **Code4Projects newsletter** on Substack for new articles and future ebook releases
- Starring the GitHub repository at github.com/sasadangelo/docker-tutorials
- Sharing the book with a colleague who is just getting started with containers

Thank you for reading.

— Salvatore D'Angelo

code4projects.org

*The best way to learn Docker is to use it every day.
Start with one container. Build from there.*